



POLITECNICO
MILANO 1863

Wrapping Linear Models from River to CapyMOA

github.com/Streaming-Data-Analytics/2025-2026_Linear-Models-in-CapyMOA

Matteo Gatti

10840092

Christian Carstens

11042297

Optional project of “Streaming Data Analytics” course

Academic Year 2025-2026

Outline

➡ 1. Context and Goals

➡ 2. Adopted procedure

➡ 3. Implementation decisions

➡ 4. Logistic Regression

➡ 5. Softmax Regression

➡ 6. River vs CapyMOA

➡ 7. Notebooks integration tutorial

➡ 8. Project structure and instructions

➡ 9. Future work

➡ 10. References





POLITECNICO
MILANO 1863

Context and Goals

Context and Goals

Context

River provides several online linear models that are not yet available in CapyMOA.

This limits the ability to:

- fairly compare models across frameworks;
- reuse well-established River implementations;
- benchmark CapyMOA against state-of-the-art online methods.

Goals

- Bridge the gap between River and CapyMOA by implementing selected linear models in the CapyMOA environment.
- Perform consistent experimental comparisons across frameworks in streaming settings.
- Make the new models usable inside CapyMOA pipelines.





POLITECNICO
MILANO 1863

Adopted procedure

Adopted procedure

➡ First Step (Individually)

- Study each model and check implementation in River
- Check the standards to follow for MOA and CappyMOA
- Implement each model and test basic metrics

➡ Second step (As a team)

- Compare the model implementation decisions with the colleague and agree on final model
- Standardize performance and metric notebooks
- CappyMOA tutorials integration



POLITECNICO
MILANO 1863

Implementation decisions

➡ Chosen Models

- Focus on classification
 - Logistic Regression
 - Softmax Regression

➡ Optimizers

- We focus on the main optimizer for this model: Stochastic Gradient Descent
 - Best proof of concept and most used optimizer for these models.
 - Open the door to more optimizers for future collaborations



POLITECNICO
MILANO 1863

Logistic Regression

Logistic regression theory

Logistic Regression is a linear model for **binary** classification. It estimates the probability that an input x belongs to the positive class.

➡ Prediction

Given an instance x , the model computes a **linear score** (a dot product between the weights of the model and the features of the instance, plus the bias):

$$z = w \cdot x + b$$

Then it converts it into a probability score using the **sigmoid** function: $\mathbb{P}(y = 1|x) = \sigma(z) = \frac{1}{1 + e^{-z}}$

The final prediction is: $\hat{y} = \begin{cases} 1 & \text{if } \mathbb{P}(y = 1|x) \geq 0.5 \\ 0 & \text{otherwise} \end{cases}$

Logistic regression theory (2)

➡ Training

The model is trained by minimizing the **binary log-loss**: $L = -y \log(p) - (1 - y) \log(1 - p)$
(where y is the true label and p is the prediction of the model, that is the probability of the positive class)

The gradient with respect to the logit z is: $\frac{\partial L}{\partial z} = p - y$

Weights and bias are updated with **Stochastic Gradient Descent (SGD)**: $\omega_i \leftarrow \omega_i - \eta \frac{\partial L}{\partial z} x_i$ and $b \leftarrow b - \eta \frac{\partial L}{\partial z}$
(in our case we will use a different learning rate hyperparameter for the bias update)

➡ Regularization

Weights update with L2 regularization: $\omega_i \leftarrow \omega_i - \eta \left(\frac{\partial L}{\partial z} x_i + \lambda_2 \omega_i \right)$

Since L1 regularization is not differentiable at zero, we cannot apply directly the standard gradient update.
In our implementation, we use a **cumulative L1 penalty**, following the same idea used in River.



MOA implementation - Prediction

→ Validation

Ensures binary input and correct feature count.

→ Initialization

Initializes weights, bias, and L1 state on first run.

→ Probability

Computes raw score (linear score) and applies sigmoid activation.

→ Output

Returns an array of length 2 (p1 is the probability of the positive class).

```
@Override
public double[] getVotesForInstance(Instance inst) {

    checkInstance(inst);

    //Initialization of the weights + bias (if needed)
    initModelIfNeeded(inst);

    // Compute the score
    double score = rawScore(inst);

    //Compute the sigmoid
    double p1 = sigmoid(score); // probability of the positive class
    double p0 = 1.0 - p1; // probability of the negative class (c0)

    double[] votes = new double[2]; // Binary classifier admits 2 classes
    votes[0] = p0; // class 0
    votes[1] = p1; // class 1
    return votes;
}
```



MOA implementation - Training

➔ Validation

Ensures binary input and correct feature count.
Moreover, ensures that L1 and L2 regularizations are not both enabled.

➔ Initialization

Initializes weights, bias, and L1 state on first run.

➔ Read Hyperparameters

Get hyperparameters values: learning rate of weights and bias, L1-term and L2-term.

➔ L1 Initialization

Initializes L1 variables in case it is enabled after model initialization.

```
@Override
public void trainOnInstanceImpl(Instance inst) {
    // Skip training if the class label is missing.
    if (inst.classIsMissing()) {
        return;
    }

    checkInstance(inst);
    checkRegularizationConf();
    initModelIfNeeded(inst);

    double lrWeights = this.weightsLearningRateOption.getValue();
    double lrBias = this.biasLearningRateOption.getValue();

    double l1 = this.l1Option.getValue();
    double l2 = this.l2Option.getValue();

    // Initialize L1 state in case L1 is enabled after model initialization.
    if (l1 != 0.0 && this.cumL1 == null) {
        this.cumL1 = new double[inst.numInputAttributes()];
        this.maxCumL1 = 0.0;
    }
}
```

MOA implementation - Training (2)

→ Target & Weight

Extracts class value (target) and instance weight (used for oversampling techniques).

→ Probability

Computes raw score (linear score) and applies sigmoid activation.

→ Gradient Clipping

Calculates loss gradient w.r.t. the logit and clips it for stability.

→ Bias Update

Performs gradient descent update on the bias term.

```
double y = inst.classValue();
double instanceWeight = inst.weight(); // Instance weight for weighted updates.

double score = rawScore(inst);
double p = sigmoid(score);

// Loss gradient term used to update both weights and bias
// Gradient of the log-loss with respect to the logit  $z = W \cdot X + b$ 
double logitGradient = (p - y) * instanceWeight;
logitGradient = clipGradient(logitGradient);

// Gradient descent update of the bias (no regularization)
this.bias -= lrBias * logitGradient;
```

MOA implementation - Training (3)

➡ Weights Update

Iterates through attributes to compute gradients and apply stochastic gradient descent (SGD) with L2 regularization.

➡ L1 Regularization

Updates cumulative L1 penalty and applies L1 regularization to weights.

```
// Gradient descent update of the weights
for (int i = 0; i < inst.numInputAttributes(); i++) {
    double x_i = inst.valueInputAttribute(i);
    double weightGradient = logitGradient * x_i; // Gradient of the log-loss w.r.t. weight w_i

    // w_i <- w_i - lr * (gradient_logLoss + l2 * w_i)
    // We perform gradient descent (we minimize the log-loss)
    this.weights[i] -= lrWeights * (weightGradient + l2 * this.weights[i]);
}

// L1 applied to weights
if (l1 != 0.0) {
    this.maxCumL1 += l1 * lrWeights;
    applyL1(inst);
}
}
```



MOA implementation - rawScore function

```
/**
 * Computes the linear score of the given instance.
 *
 * @param inst the input instance.
 * @return the linear score  $z = w \cdot x + b$ .
 */
protected double rawScore(Instance inst) { 2 usages
    double score = this.bias;
    for (int i = 0; i < inst.numInputAttributes(); i++) {
        score += this.weights[i] * inst.valueInputAttribute(i); // dot_product(w,x) + b
    }
    return score;
}
```



MOA implementation - sigmoid function

```
/**
 * Computes a numerically stable sigmoid function (to prevent overflow).
 *
 * @param z the input value (linear score).
 * @return the sigmoid of {@code z}.
 */
protected double sigmoid(double z) { 2 usages
    if (z >= 0.0) {
        // Standard sigmoid form
        double expNeg = Math.exp(-z);
        return 1.0 / (1.0 + expNeg);
    } else {
        // Alternative sigmoid formula (equivalent to the standard one)
        double expPos = Math.exp(z);
        return expPos / (1.0 + expPos);
    }
}
```



MOA implementation - applyL1 function

```
/**
 * Applies River-style cumulative L1 penalty to the weights corresponding to the non-zero features of the current instance.
 * This must be applied after the standard weights update. Look at river/linear_model/base.py (class GLM)
 *
 * @param inst the instance used to train the model.
 */
protected void applyL1(Instance inst) { 1 usage

    for (int i = 0; i < inst.numInputAttributes(); i++) {
        double x_i = inst.valueInputAttribute(i);

        // Skip zero-valued features.
        if (x_i == 0.0) {
            continue;
        }

        double curr_w_i = this.weights[i]; // Value of the weight before L1 regularization

        if (curr_w_i > 0.0) {
            this.weights[i] = Math.max(0.0, curr_w_i - (this.maxCumL1 + this.cumL1[i]));
        } else if (curr_w_i < 0.0) {
            this.weights[i] = Math.min(0.0, curr_w_i + (this.maxCumL1 - this.cumL1[i]));
        }

        // Update cumulative correction for weight i
        this.cumL1[i] += this.weights[i] - curr_w_i;
    }
}
```



Implementation comparisons - Prediction

MOA

```
@Override
public double[] getVotesForInstance(Instance inst) {

    checkInstance(inst);

    //Initialization of the weights + bias (if needed)
    initModelIfNeeded(inst);

    // Compute the score
    double score = rawScore(inst);

    //Compute the sigmoid
    double p1 = sigmoid(score); // probability of the positive class
    double p0 = 1.0 - p1; // probability of the negative class (c0)

    double[] votes = new double[2]; // Binary classifier admits 2 classes
    votes[0] = p0; // class 0
    votes[1] = p1; // class 1
    return votes;
}
```

River

→ *river/linear_model/log_reg.py*

```
def predict_proba_one(self, x):
    p = self.loss.mean_func(self._raw_dot_one(x))
    return {False: 1.0 - p, True: p}
```

→ *river/optim/losses.py*

```
def mean_func(self, y_pred):
    if isinstance(y_pred, np.ndarray):
        return 1.0 / (1.0 + np.exp(-y_pred))
    return utils.math.sigmoid(y_pred)
```



Implementation comparisons - Training

MOA

```
double y = inst.classValue();
double instanceWeight = inst.weight(); // Instance weight for weighted updates.

double score = rawScore(inst);
double p = sigmoid(score);

// Loss gradient term used to update both weights and bias
// Gradient of the log-loss with respect to the logit  $z = W \cdot X + b$ 
double logitGradient = (p - y) * instanceWeight;
logitGradient = clipGradient(logitGradient);

// Gradient descent update of the bias (no regularization)
this.bias -= lrBias * logitGradient;
```

River

➡ *river/linear_model/base.py*

```
def learn_one(self, x, y, w=1.0) -> None:
    with self._learn_mode(x):
        self._fit(x, y, w, get_grad=self._eval_gradient_one)

def _eval_gradient_one(self, x: dict, y: float, w: float) -> tuple[dict, float]:
    loss_gradient = self.loss.gradient(y_true=y, y_pred=self._raw_dot_one(x))
    loss_gradient *= w
    loss_gradient = float(
        utils.math.clamp(loss_gradient, -self.clip_gradient, self.clip_gradient)
    )

    if self.l2:
        return (
            utils.VectorDict(x) * loss_gradient + self.l2 * self._weights,
            loss_gradient,
        )

    return (utils.VectorDict(x) * loss_gradient, loss_gradient)

def _fit(self, x, y, w, get_grad):
    # Some optimizers need to do something before a prediction is made
    self.optimizer.look_ahead(w=self._weights)

    # Calculate the gradient
    gradient, loss_gradient = get_grad(x, y, w)

    # Update the intercept
    self.intercept -= self._get_intercept_update(loss_gradient)
```



Implementation comparisons - Training (2)

MOA

```
// Gradient descent update of the weights
for (int i = 0; i < inst.numInputAttributes(); i++) {
    double x_i = inst.valueInputAttribute(i);
    double weightGradient = logitGradient * x_i; // Gradient of the log-loss w.r.t. weight w_i

    // w_i <- w_i - lr * (gradient_logLoss + l2 * w_i)
    // We perform gradient descent (we minimize the log-loss)
    this.weights[i] -= lrWeights * (weightGradient + l2 * this.weights[i]);
}

// L1 applied to weights
if (l1 != 0.0) {
    this.maxCumL1 += l1 * lrWeights;
    applyL1(inst);
}
}
```

River

➔ *river/linear_model/base.py*
(***_fit*** function)

```
# Update the weights
self.optimizer.step(w=self._weights, g=gradient)

# Apply L1 cumulative penalty if applicable
if self.l1 != 0.0:
    # This should be called after the learning_rate update in case of adaptive learning rate
    self.max_cum_l1 = self.max_cum_l1 + self.l1 * self.optimizer.learning_rate

    self._update_weights(x)
```



Implementation comparisons - L1 regularization

MOA

```
/**
 * Applies River-style cumulative L1 penalty to the weights corresponding to the non-zero features of the current instance.
 * This must be applied after the standard weights update. Look at river/linear_model/base.py (class GLM)
 *
 * @param inst the instance used to train the model.
 */
protected void applyL1(Instance inst) { 1 usage

    for (int i = 0; i < inst.numInputAttributes(); i++) {
        double x_i = inst.valueInputAttribute(i);

        // Skip zero-valued features.
        if (x_i == 0.0) {
            continue;
        }

        double curr_w_i = this.weights[i]; // Value of the weight before L1 regularization

        if (curr_w_i > 0.0) {
            this.weights[i] = Math.max(0.0, curr_w_i - (this.maxCumL1 + this.cumL1[i]));
        } else if (curr_w_i < 0.0) {
            this.weights[i] = Math.min(0.0, curr_w_i + (this.maxCumL1 - this.cumL1[i]));
        }

        // Update cumulative correction for weight i
        this.cumL1[i] += this.weights[i] - curr_w_i;
    }
}
```

River

➔ *river/linear_model/base.py*

```
def _update_weights(self, x):
    # L1 cumulative penalty helper

    for j, xj in x.items():
        wj_temp = self._weights[j]

        if wj_temp > 0:
            self._weights[j] = max(0, wj_temp - (self.max_cum_l1 + self.cum_l1[j]))
        elif wj_temp < 0:
            self._weights[j] = min(0, wj_temp + (self.max_cum_l1 - self.cum_l1[j]))
        else:
            self._weights[j] = wj_temp

    # Update the penalty state of the estimator
    self.cum_l1[j] = self.cum_l1[j] + (self._weights[j] - wj_temp)
```



Python wrapper - CappyMOA

➡ Input Parameters

Defines the input for creating the model. They include hyperparameters, the seed and the schema (input data source).

➡ Mapping

Defines the mapping between the Python arguments and the corresponding Java CLI options.

➡ Model creation

Defines the configuration string to be processed by Java and instantiates the model.

```
class LogisticRegression(MOAClassifier):

    def __init__(
        self,
        schema: Schema | None = None,
        random_seed: int = 0, # not used
        learning_rate: float = 0.01,
        bias_learning_rate: float = 0.01,
        l1_penalty: float = 0.0,
        l2_penalty: float = 0.0,
        clip_gradient: float = 1e12,
        bias_init: float = 0.0
    ):

        mapping = {
            "learning_rate": "-r",
            "bias_learning_rate": "-b",
            "l1_penalty": "-l",
            "l2_penalty": "-q",
            "clip_gradient": "-c",
            "bias_init": "-i"
        }

        config_str = build_cli_str_from_mapping_and_locals(mapping, locals())

        super(LogisticRegression, self).__init__(
            moa_learner=moa_classifiers.LogisticRegression,
            schema=schema,
            CLI=config_str,
            random_seed=random_seed,
        )

    def __str__(self):
        return "LogisticRegression"
```



POLITECNICO
MILANO 1863

Softmax Regression

Softmax regression theory

Softmax Regression (or Multinomial Logistic Regression) is a linear model for **multi-class** classification. It estimates the probability that an input x belongs to each of the K possible classes.

➡ Prediction

Given an instance x , the model computes a **linear score** for every class k (a dot product between the weights of the model and the features of the instance): $z = w \cdot x + b$

Then it converts it into a probability score using the **softmax** function: $\mathbb{P}(y = k|x) = \sigma(z) = \frac{e^{z_k}}{\sum_{j=1}^K e^{z_j}}$

The final prediction is: $\hat{y} = \operatorname{argmax}_k \mathbb{P}(y = k|x)$



Softmax regression theory (2)

➡ Training

The model is trained by minimizing the **categorical cross-entropy loss**:
$$L = - \sum_{k=1}^K y_k \log(p_k)$$

And the gradient:
$$\frac{\partial L}{\partial z_k} = p_k - y_k$$

Weights and bias are updated with **Stochastic Gradient Descent (SGD)**:

(in our case we will use a different learning rate hyperparameter for the bias update)

$$\begin{aligned}\omega_{k,i} &\leftarrow \omega_{k,i} - \eta \frac{\partial L}{\partial z_k} x_i \\ b_k &\leftarrow b_k - \eta \frac{\partial L}{\partial z_k}\end{aligned}$$

➡ Regularization

Weights update with L2 regularization:
$$\omega_{k,i} \leftarrow \omega_{k,i} - \eta \left(\frac{\partial L}{\partial z_k} x_i + \lambda_2 \omega_{k,i} \right)$$

MOA implementation - Prediction

➡ Initialization

Initializes weights and bias on first run.

➡ Probability

Computes logits and applies softmax activation.

➡ Output

Returns an array of length k (number of classes)

```
@Override
public double[] getVotesForInstance(Instance inst) {
    initModelIfNeeded(inst);

    double[] logits = computeLogits(inst);
    double[] probabilities = softmax(logits);

    // If instance expects less classes than we have, return the expected number
    if (inst.numClasses() < probabilities.length) {
        double[] returnedProbabilities = new double[inst.numClasses()];
        System.arraycopy(probabilities, 0, returnedProbabilities, 0, inst.numClasses());
        return returnedProbabilities;
    }

    return probabilities;
}
```

MOA implementation - Training

→ Validation

Check real class is not missing for the instance

→ Initialization

Initializes weights and bias

→ Read Hyperparameters

Get hyperparameters values: learning rate of weights and bias and L2-term.

```
@Override
public void trainOnInstanceImpl(Instance inst) {
    if (inst.classIsMissing()) {
        return;
    }

    int targetClass = (int) inst.classValue();
    int instNumClasses = inst.numClasses();

    prepareForClasses(Math.max(instNumClasses, targetClass + 1));

    double lrBias = this.biasLearningRateOption.getValue();
    double lr = this.weightsLearningRateOption.getValue();
    double l2 = this.l2Option.getValue();
}
```

MOA implementation - Training (2)

→ Compute logits

For each class, we compute the logit and keep the max

→ Compute Probabilities

We use max subtraction trick to avoid overflow in the exponential

```
// 1. Forward pass: compute logits
double[] logits = computeLogits(inst);
// 2. Compute probabilities using Softmax
double[] probabilities = softmax(logits);
```



MOA implementation - Training (3)

➡ Weights and Bias Update

Iterates through attributes to compute gradients and apply stochastic gradient descent (SGD) with L2 regularization.

```
// 3. Update weights and biases for each class
for (int k = 0; k < this.numClasses; k++) {
    double target = (k == targetClass) ? 1.0 : 0.0;
    double lossGradient = (probabilities[k] - target) * inst.weight();

    // Gradient descent update of the bias (no regularization)
    this.biases[k] -= lrBias * lossGradient;

    double[] classWeights = this.weights[k];

    // Update feature weights
    for (int i = 0; i < inst.numValues(); i++) {
        int idx = inst.index(i);
        if (idx != inst.classIndex() && !inst.isMissingSparse(i)) {
            // Resize weights array if necessary
            if (idx >= classWeights.length) {
                int newSize = Math.max(classWeights.length * 2, idx + 1);
                double[] newW = new double[newSize];
                System.arraycopy(classWeights, 0, newW, 0, classWeights.length);
                classWeights = newW;
                this.weights[k] = classWeights;
            }

            double xi = inst.valueSparse(i);
            double currentWeight = classWeights[idx];

            // L2 Regularization
            if (l2 > 0.0) {
                currentWeight *= (1.0 - lr * l2);
            }

            // SGD Update
            currentWeight -= lr * lossGradient * xi;
            classWeights[idx] = currentWeight;
        }
    }
}
```



MOA implementation - Logit function

```
/**
 * Computes raw score (logit) for a specific class.
 */
protected double computeLogit(Instance inst, int classIndex) { 1 usage
    if (classIndex >= this.numClasses) {
        return 0.0;
    }

    double result = this.biases[classIndex];
    double[] classWeights = this.weights[classIndex];

    for (int i = 0; i < inst.numValues(); i++) {
        int idx = inst.index(i);
        if (idx != inst.classIndex() && !inst.isMissingSparse(i)) {
            if (idx < classWeights.length) {
                result += inst.valueSparse(i) * classWeights[idx];
            }
        }
    }

    return result;
}
```



MOA implementation - Softmax function

```
private double[] softmax(double[] logits) { 2 usages
    double maxLogit = Double.NEGATIVE_INFINITY;
    for (double l : logits) {
        if (l > maxLogit) {
            maxLogit = l;
        }
    }

    double[] probabilities = new double[logits.length];
    double sumExp = 0.0;
    for (int k = 0; k < logits.length; k++) {
        probabilities[k] = Math.exp(logits[k] - maxLogit);
        sumExp += probabilities[k];
    }
    for (int k = 0; k < logits.length; k++) {
        probabilities[k] /= sumExp;
    }
    return probabilities;
}
```



Implementation comparisons - Prediction

MOA

```
private double[] softmax(double[] logits) { 2 usages
    double maxLogit = Double.NEGATIVE_INFINITY;
    for (double l : logits) {
        if (l > maxLogit) {
            maxLogit = l;
        }
    }

    double[] probabilities = new double[logits.length];
    double sumExp = 0.0;
    for (int k = 0; k < logits.length; k++) {
        probabilities[k] = Math.exp(logits[k] - maxLogit);
        sumExp += probabilities[k];
    }
    for (int k = 0; k < logits.length; k++) {
        probabilities[k] /= sumExp;
    }
    return probabilities;
}
```

River

→ *river/linear_model/log_reg.py*

```
def predict_proba_one(self, x):
    return utils.math.softmax(
        {label: utils.math.dot(weights, x) for label, weights in self.weights.items()}
    )
```

→ *river/utils/math.py*

```
def softmax(y_pred: MutableMapping[Any, float]) -> MutableMapping[Any, float]:
    """Normalizes a dictionary of predicted probabilities, in-place.

    Parameters
    -----
    y_pred
    """

    if not y_pred:
        return y_pred

    maximum = max(y_pred.values())
    total = 0.0

    for c, p in y_pred.items():
        y_pred[c] = math.exp(p - maximum)
        total += y_pred[c]

    for c in y_pred:
        y_pred[c] /= total

    return y_pred
```



Implementation comparisons - Training

MOA

```
// 3. Update weights and biases for each class
for (int k = 0; k < this.numClasses; k++) {
    double target = (k == targetClass) ? 1.0 : 0.0;
    double lossGradient = (probabilities[k] - target) * inst.weight();

    // Gradient descent update of the bias (no regularization)
    this.biases[k] -= lrBias * lossGradient;

    double[] classWeights = this.weights[k];

    // Update feature weights
    for (int i = 0; i < inst.numValues(); i++) {
        int idx = inst.index(i);
        if (idx != inst.classIndex() && !inst.isMissingSparse(i)) {
            // Resize weights array if necessary
            if (idx >= classWeights.length) {
                int newSize = Math.max(classWeights.length * 2, idx + 1);
                double[] newW = new double[newSize];
                System.arraycopy(classWeights, 0, newW, 0, classWeights.length);
                classWeights = newW;
                this.weights[k] = classWeights;
            }

            double xi = inst.valueSparse(i);
            double currentWeight = classWeights[idx];

            // L2 Regularization
            if (l2 > 0.0) {
                currentWeight *= (1.0 - lr * l2);
            }

            // SGD Update
            currentWeight -= lr * lossGradient * xi;
            classWeights[idx] = currentWeight;
        }
    }
}
```

River

➡ *river/linear_model/softmax.py*

```
def learn_one(self, x, y):
    # Some optimizers need to do something before a prediction is made
    for label, weights in self.weights.items():
        self.optimizers[label].look_ahead(w=weights)

    # Make a prediction for the given features
    y_pred = self.predict_proba_one(x)

    # Compute the gradient of the loss w.r.t. each label
    loss_gradients = self.loss.gradient(y_true=y, y_pred=y_pred)

    for label, loss in loss_gradients.items():
        # Compute the gradient w.r.t. each feature
        weights = self.weights[label]
        gradient = {i: xi * loss + self.l2 * weights.get(i, 0) for i, xi in x.items()}
        self.weights[label] = self.optimizers[label].step(w=weights, g=gradient)
```



Python wrapper - CapyMOA

➡ Input Parameters

Defines the input for creating the model. They include hyperparameters, the seed (unused due softmax is deterministic) and the schema.

➡ Mapping

Defines the mapping between the Python arguments and the corresponding Java CLI options.

➡ Model creation

Defines the configuration string to be processed by Java and instantiates the model.

```
class SoftmaxRegression(MOAClassifier):
    def __init__(
        self,
        schema: Schema | None = None,
        random_seed: int = 0, # not used
        learning_rate: float = 0.01,
        bias_learning_rate: float = 0.01,
        l2_penalty: float = 0.0,
        bias_init: float = 0.0,
    ):
        """Construct Softmax Regression.

        :param schema: Stream schema.
        :param random_seed: Seed for reproducibility.
        :param learning_rate: The learning rate for the SGD optimizer.
        :param bias_learning_rate: The learning rate for the intercept (bias).
        :param l2_penalty: The L2 regularization weight.
        :param bias_init: Initial value for the intercept (bias).
        """
        mapping = {
            "learning_rate": "-r",
            "bias_learning_rate": "-b",
            "l2_penalty": "-l",
            "bias_init": "-i",
        }

        config_str = build_cli_str_from_mapping_and_locals(mapping, locals())

        # Initialize the parent MOAClassifier
        super(SoftmaxRegression, self).__init__(
            moa_learner=moa_classifiers.SoftmaxRegression,
            schema=schema,
            CLI=config_str,
            random_seed=random_seed,
        )

    def __str__(self):
        return "SoftmaxRegression"
```



POLITECNICO
MILANO 1863

River vs CapyMOA

Cross-framework comparison

This section compares the predictive behavior of the same linear models across CapyMOA and River.

Relevant notebooks:

➡ *log_reg_cross_framework_comparison.ipynb*

➡ *softmax_reg_cross_framework_comparison.ipynb*

The compared metrics are:

➡ **Accuracy**

➡ **F1-score**

➡ **Recall**

➡ **Precision**

For Softmax Regression, we used **macro-averaged** Precision, Recall, and F1-score, computed as the arithmetic mean of the per-class scores, to give equal importance to all classes.

We also included **confusion matrices** to better interpret the results.

Cross-framework comparison (2)

All experiments are carried out in a **prequential evaluation** setting, using a custom loop to collect the models' predictions and pass them to an external library used for metric computation.

We decided to compute the metrics with an external library, **scikit-learn**, instead of relying on framework-specific implementations. This makes the comparison fair and reliable.

The same streams are evaluated in both frameworks, converting CapyMOA streams into a River-compatible format.

For Softmax Regression, since River does not model the bias explicitly, we add a constant feature equal to 1 to all samples to simulate the intercept term. In our CapyMOA implementation, instead, the bias is handled explicitly by the model.

We also compared the models under different hyperparameter settings to verify whether they behave consistently in those cases as well.



Cross-framework comparison - Logistic Regression

Electricity dataset

```
from capymoa.datasets import Electricity
evaluateStream(Electricity)
```

Metric	CapyMOA	River	Delta
Accuracy	69.93%	69.93%	+0.00%
F1	75.02%	75.02%	+0.00%
Precision	71.86%	71.86%	+0.00%
Recall	78.48%	78.48%	+0.00%

DriftStream dataset generator

```
from capymoa.stream.drift import DriftStream, AbruptDrift, GradualDrift
from capymoa.stream.generator import SEA

def make_stream():
    return DriftStream(
        stream=[
            SEA(function=1, # concept A
                AbruptDrift(position=5000),
                SEA(function=3, # concept B
                    GradualDrift(position=10000, width=3000),
                    SEA(function=1, # concept A again

```

Metric	CapyMOA	River	Delta
Accuracy	83.26%	83.26%	+0.00%
F1	87.40%	87.40%	+0.00%
Precision	85.01%	85.01%	+0.00%
Recall	89.93%	89.93%	+0.00%

CapyMOA and River produce **identical classification metrics** in both basic and drifting settings.

Cross-framework comparison - Softmax Regression

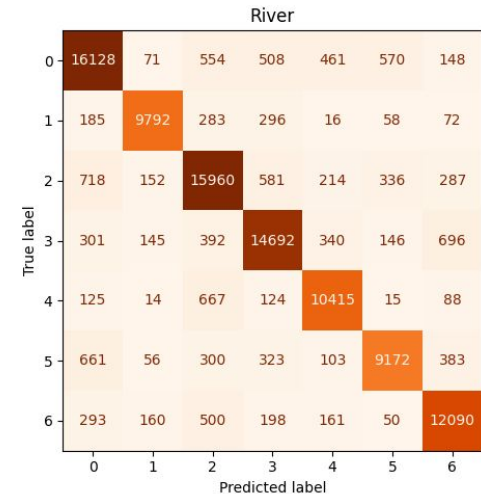
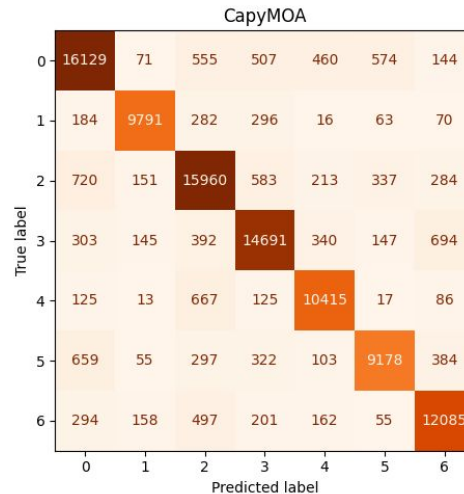
CapyMOA dataset generator

```
from capymoa.stream.generator import RandomRBFGenerator

def make_stream():
    return RandomRBFGenerator(
        number_of_classes=7,
        number_of_attributes=50,
        number_of_centroids=100,
        model_random_seed=SEED
    )

evaluateStream(make_stream)
```

Metric	CapyMOA	River	Delta
Accuracy	88.25%	88.25%	+0.00%
F1	88.50%	88.50%	-0.00%
Precision	88.66%	88.67%	-0.00%
Recall	88.37%	88.37%	+0.00%



CapyMOA and River produce **identical** classification metrics.
Moreover their confusion matrices closely match.

Cross-framework comparison - Softmax Regression (2)

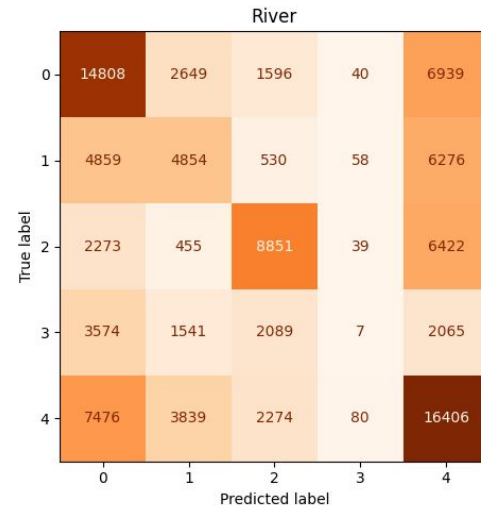
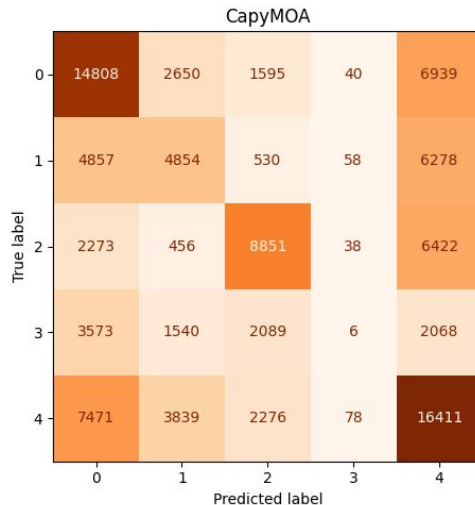
CapyMOA drifting dataset generator

```
from capymoa.stream.generator import RandomRBFGeneratorDrift

def make_stream():
    return RandomRBFGeneratorDrift(
        number_of_classes=5,
        number_of_attributes=10,
        number_of_centroids=50,
        number_of_drifting_centroids=2,
        magnitude_of_change=0.5,
    )

evaluateStream(make_stream())
```

Metric	CapyMOA	River	Delta
Accuracy	44.93%	44.93%	+0.00%
F1	36.78%	36.79%	-0.00%
Precision	36.95%	37.03%	-0.08%
Recall	37.97%	37.97%	+0.00%



CapyMOA and River produce **nearly identical** classification metrics.

Computational Performance Analysis

This section compares the **computational efficiency** of the CappyMOA and River implementations.

Relevant notebooks:

➡ `log_reg_computational_performance_analysis.ipynb`

➡ `softmax_reg_computational_performance_analysis.ipynb`

The compared metrics are:

➡ training and inference **time**
(measured with the *time* library)

➡ **memory** usage
(measured with the *tracemalloc* library,
that tracks the peak allocated memory)

Computational Performance Analysis (2)

Evaluation setting

In this section as well, the experiments are run in a **prequential setting**.

In this case we used the native prequential evaluation functions for both CapyMOA and River, since they are more efficient than a custom evaluation loop.

Note on classification metrics

Although some classification metrics are still shown in these notebooks, they are **not directly comparable** here, because they are computed using the native metric implementations of each framework

If the underlying metric computation differs, then the values cannot be used for a strict cross-framework comparison.



Computational Performance Analysis - Logistic Regression

RandomRBFGenerator

```
from capymoa.stream.generator import RandomRBFGenerator

def make_stream():
    return RandomRBFGenerator(
        number_of_classes=2,
        number_of_attributes=50,
        number_of_centroids=100,
        model_random_seed=SEED
    )

evaluateStream(make_stream())
```

Metric	CapyMOA	River	Delta
Time (s)	0.2217	18.2704	-18.0487
Memory (MB)	0.1518	0.1561	-0.0043
Accuracy	85.28%	85.28%	+0.00%
F1	84.75%	81.59%	+3.16%

HyperPlaneClassification dataset

```
from capymoa.stream.generator import HyperPlaneClassification

def make_stream():
    return HyperPlaneClassification(
        instance_random_seed=SEED,
        number_of_classes=2,
        number_of_attributes=10,
        number_of_drifting_attributes=2,
        magnitude_of_change=0.0,
        noise_percentage=5,
        sigma_percentage=10,
    )

evaluateStream(make_stream())
```

Metric	CapyMOA	River	Delta
Time (s)	0.0497	9.5030	-9.4533
Memory (MB)	0.0107	0.1497	-0.1390
Accuracy	89.87%	89.87%	-0.00%
F1	89.94%	90.18%	-0.24%

CapyMOA achieves significantly **lower execution time** and slightly **lower memory usage** than River in both settings.

Computational Performance Analysis - Softmax Regression

WaveformGenerator

```
from capymoa.stream.generator import WaveformGenerator

def make_stream():
    return WaveformGenerator(instance_random_seed=SEED)

evaluateStream(make_stream())
```

Metric	CapyMOA	River	Delta
Time (s)	0.3753	26.0814	-25.7061
Memory (MB)	0.1519	0.1582	-0.0063
Accuracy	83.29%	83.30%	-0.00%
F1	83.28%	84.36%	-1.09%

RTG_2abrupt dataset

```
from capymoa.datasets import RTG_2abrupt

evaluateStream(RTG_2abrupt)
```

Metric	CapyMOA	River	Delta
Time (s)	2.1972	40.6195	-38.4223
Memory (MB)	0.1519	0.1640	-0.0121
Accuracy	68.11%	68.14%	-0.03%
F1	50.01%	47.56%	+2.44%

CapyMOA is consistently **faster** and slightly **more memory-efficient** than River in both settings.



POLITECNICO
MILANO 1863

Notebooks integration tutorial

Integration

The objective of this section was to show that the implemented models work correctly inside the native CapyMOA workflow and can be used consistently in the same educational and experimental environment provided by the framework.

Relevant notebooks:

➡ *00_getting_started.ipynb*

➡ *01_evaluation.ipynb*

We integrated our models into these notebooks in two ways:

- we inserted code for our models directly inside existing tutorial cells, adapting the original workflow to include the new learners;
- we created dedicated cells for our implemented models by reusing the original tutorial code and replacing the existing learner with our custom model.

We also added comparison cells to better analyze the differences between the original notebook models and our implementations, using both textual comments and graphical outputs.

Integration - Logistic Regression

Code

Duplicated cell for our LogisticRegression

```
from capymoa.evaluation import prequential_evaluation
# from capymoa.classifier import HoeffdingTree

# ht = HoeffdingTree(schema=elec_stream.get_schema(), grace_period=50)
log_reg = LogisticRegression(
    schema=elec_stream.get_schema(),
)

# Obtain the results from the high-level function.
# Note that we need to specify a window_size as we obtain both windowed and cumulative results.
# The results from a high-level evaluation function are represented as a PrequentialResults object.
# results_ht = prequential_evaluation(stream=elec_stream, learner=ht, window_size=4500)
results_log_reg = prequential_evaluation(stream=elec_stream, learner=log_reg, window_size=4500)

print(
    f"Cumulative accuracy = {results_log_reg.cumulative.accuracy()}, wall-clock time: {results_log_reg.wallclock()}"
)

# The windowed results are conveniently stored in a pandas DataFrame.
display(results_log_reg.windowed.metrics_per_window())
```

✓ 0.4s

Output

Cumulative accuracy = 69.92849576271186, wall-clock time: 0.40457987785339355

	instances	accuracy	kappa	kappa_t	kappa_m	f1_score	f1_score_0	f1_score_1	precision	precision_0	precision_1	recall	recall_0	recall_1
0	4500.0	65.488889	23.046684	-113.911846	12.061155	62.127162	47.050001	74.402505	63.482461	59.329321	67.635601	60.828522	38.983051	82.673993
1	9000.0	64.933333	29.076243	-109.006623	23.694391	64.562399	60.628743	68.389423	64.654277	62.628866	66.679688	64.470781	58.752418	70.189145
2	13500.0	69.422222	38.721949	-108.169440	36.118849	69.361597	67.985109	70.735857	69.368565	68.271028	70.466102	69.354631	67.701576	71.007686
3	18000.0	69.933333	37.116175	-108.796296	28.298887	68.684255	61.638787	75.278641	69.154230	66.280488	72.027972	68.220625	57.604663	78.836586
4	22500.0	73.755556	42.692925	-64.484680	33.011912	71.747404	62.376553	79.849855	73.026113	71.148256	74.903969	70.512707	55.530346	85.495068
5	27000.0	72.088889	41.783094	-119.580420	32.834225	70.956289	64.837626	76.860722	71.295428	68.037603	74.553252	70.620362	61.925134	79.315589
6	31500.0	71.866667	39.396028	-131.021898	27.780947	69.824598	61.331704	77.890325	70.433270	66.009204	74.857335	69.226357	57.273246	81.179469
7	36000.0	67.555556	30.285460	-145.378151	18.843802	65.337114	55.378973	74.511173	66.002986	61.507128	70.498844	64.684543	50.361312	79.007775
8	40500.0	72.422222	43.193747	-86.056972	36.391594	71.682691	66.540847	76.545077	72.022303	70.193402	73.851204	71.346267	63.249616	78.442919
9	45000.0	71.866667	43.107783	-80.857143	43.075540	71.570842	68.507463	74.578313	71.662019	70.040692	73.283346	71.479896	67.039922	75.919869
10	45312.0	71.933333	43.052700	-83.309144	42.695100	71.557589	68.017220	74.995050	71.699704	70.130548	73.268859	71.416037	66.027532	76.804542



Integration - Softmax Regression

Code

Duplicated cell for our SoftmaxRegression

```
# from capymoa.classifier import OnlineBagging
from capymoa.stream.generator import SEA
from capymoa.stream.drift import AbruptDrift, GradualDrift, DriftStream

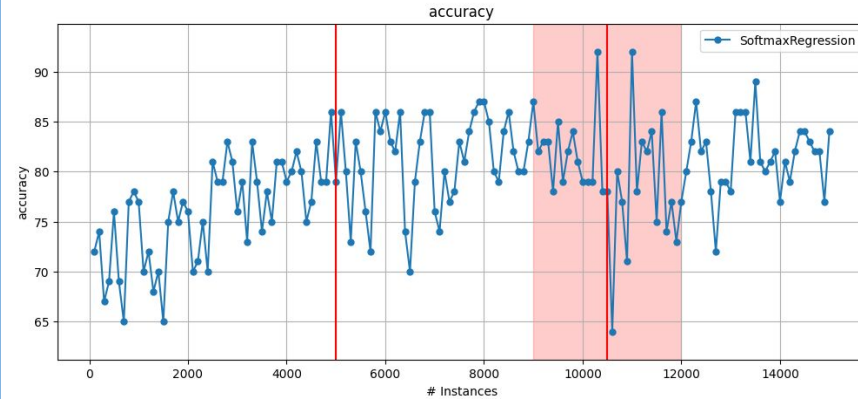
# Generating a synthetic stream with 1 abrupt drift and 1 gradual drift.
stream_sea2drift = DriftStream(
    stream=[
        SEA(function=1),
        AbruptDrift(position=5000),
        SEA(function=3),
        GradualDrift(start=9000, end=12000),
        SEA(function=1),
    ]
)

# OB = OnlineBagging(schema=stream_sea2drift.get_schema(), ensemble_size=10)
SOFTMAX_REG = SoftmaxRegression(schema=stream_sea2drift.get_schema())

# Since this is a synthetic stream, max_instances is needed to determine the amount of instances to be generated.
...
results_sea2drift_OB = sequential_evaluation(
    stream=stream_sea2drift, learner=OB, window_size=100, max_instances=15000
)
...
results_sea2drift_SOFTMAX_REG = sequential_evaluation(
    stream=stream_sea2drift, learner=SOFTMAX_REG, window_size=100, max_instances=15000
)

plot_windowed_results(results_sea2drift_SOFTMAX_REG, metric="accuracy")
```

Output





POLITECNICO
MILANO 1863

Project structure and instructions

Project structure

```
2025-2026_Linear-Models-in-CapyMOA/
```

```
|— INSTRUCTIONS.md
|— README.md
|— requirements.txt
|— capymoa/
|   |— notebooks/           # Tutorial notebooks for CapyMOA
|   |— src/capymoa/classifier/ # CapyMOA Python wrappers for the linear models
|— capymoa_vs_river/       # Jupyter notebooks for cross-framework comparison and performance analysis
|— moa/
|   |— moa/src/main/java/moa/classifiers/ # Custom MOA Java implementations (Logistic & Softmax Regression)
```



How to run the notebooks

➡ Step 1: Build the Custom MOA Project

- Clone the MOA repository: `git clone https://github.com/Waikato/moa.git`
- Copy our custom models (`LogisticRegression.java` and `SoftmaxRegression.java`) into the folder `/moa/src/main/java/moa/classifiers` of the MOA repository
- Build the MOA library: `mvn clean package -DskipTests`
- Move the generated jar file into the root directory of the project

➡ Step 2: Set Up the Python Virtual Environment

- Create a virtual environment: `python -m venv .venv`
- Activate the virtual environment

How to run the notebooks (2)

➡ Step 3: Install Python Dependencies

- Install the required python libraries specified in the requirement file:
pip install --upgrade pip
pip install -r requirements.txt

➡ Step 4: Run the notebooks (with Jupyter)

- Start the jupyter server: *jupyter notebook*
- Run the notebooks





POLITECNICO
MILANO 1863

Future work

Future work

Main objective:

➡ prepare the project for a possible **integration** into the CapyMOA open-source library

Based on this project, new features could be developed:

➡ new **optimizers**: AdaGrad, Adam, AdaMax, among others

➡ new **linear algorithms** besides classification: Linear Regression





POLITECNICO
MILANO 1863

References

References

- ➡ CopyMOA: <https://capymoa.org/>
- ➡ River: <https://riverml.xyz/latest/>
- ➡ MOA: <https://moa.cms.waikato.ac.nz/>
- ➡ Scikit-Learn: <https://scikit-learn.org/stable/>

